DevBoard — Full-Stack Project Guide

> A personal project & task management platform for developers.
> Built with: **Node.js/Express · PostgreSQL · Redis · Session Auth · React · Docker**

1. Project Overview

DevBoard is a full-stack task and project management web application tailored for developers. It is the developer equivalent of a lightweight Trello — users can create projects, manage tasks with priorities and statuses, and track their productivity through a dashboard. The project is intentionally scoped to be complex enough to showcase real engineering decisions (caching, session auth, role-based access, REST API design) without becoming unmanageable for a solo developer.

Why it impresses recruiters:

- Demonstrates a complete, production-realistic stack
- Uses Redis correctly (not just as a talking point)
- Shows SQL skills beyond basic queries
- One command to run everything via Docker Compose
- Live demo-able in under 30 seconds

2. Tech Stack

Layer	Technology	Purpose
Backend framework	Node.js + Express.js	REST API server
Primary database	PostgreSQL 16	Persistent relational data
Cache / Session store	Redis 7	Session storage, project list caching
Authentication	express-session + connect-redis	Cookie-based session auth
Password hashing	bcrypt	Secure credential storage
Security headers	helmet + cors	Production-grade HTTP security
Frontend	React (Vite)	SPA user interface
HTTP client	Axios	API calls with cookie support
Containerization	Docker + Docker Compose	One-command local dev environment
Deployment	Railway / Render	Cloud hosting (Docker-compatible)

3. Features

3.1 Authentication & Session Management

- User registration with email + password (bcrypt hashed)
- Login / Logout with `express-session` stored in Redis
- `req.session.regenerate()` on login to prevent session fixation attacks
- `httpOnly`, `secure`, `sameSite` cookie flags for security
- "Remember me" toggle extends session TTL from 24h to 7 days
- `GET /auth/me` endpoint returns current user from active session
- Instant server-side logout: destroys Redis session key on `POST /auth/logout`

3.2 Projects

- Create, edit, delete projects
- Fields: name, description, hex color label, status (`active` / `archived`)
- Projects list cached in Redis using cache-aside pattern
  - Cache key: `user:{userId}:projects`
  - Invalidated on any create/update/delete mutation
- Archive a project (soft hide) without deleting tasks

3.3 Tasks

- Full CRUD within each project
- Fields: title, description, priority (`low` / `medium` / `high`), status (`todo` / `in-progress` / `done`), due date
- Filter tasks by status or priority (client-side filtering)
- Mark task as complete with a single click
- Overdue detection: tasks past due date and not `done` are flagged
- Tasks ordered by priority then due date

3.4 Dashboard

- Summary cards: active projects count, tasks due today, tasks completed this week
- Activity feed: "You completed *Fix login bug* in *DevBoard* — 2 hours ago"
- All data from PostgreSQL aggregation queries — no frontend calculation
- Dashboard data cached in Redis for 60 seconds (TTL-based, not event-invalidated)

3.5 User Profile

- Update display name
- Initials-based avatar (auto-generated from display name, no file upload)
- Stats: total projects, tasks completed all-time, tasks currently overdue

3.6 Role-Based Access Control

- Two roles: `admin` and `member`
- `admin`: can delete any project, access user list, change user roles
- `member`: can only manage their own projects and tasks
- Role stored in session (`req.session.role`), validated by `requireRole` middleware on protected routes

4. Project Structure

```
devboard/
├── backend/
│   ├── src/
│   │   ├── config/
│   │   │   ├── db.js             # PostgreSQL pool (pg)
│   │   │   └── redis.js          # Redis client (redis v4)
│   │   ├── middlewares/
│   │   │   ├── requireAuth.js    # Blocks unauthenticated requests
│   │   │   └── requireRole.js    # Role-based access guard
│   │   ├── routes/
│   │   │   ├── auth.js           # /auth/register, /auth/login, /auth/logout, /auth/
│   │   │   ├── projects.js       # /projects CRUD
│   │   │   ├── tasks.js          # /projects/:id/tasks CRUD
│   │   │   ├── dashboard.js      # /dashboard summary + activity
│   │   │   └── users.js          # /users (admin only)
│   │   └── app.js                # Express app setup
│   ├── .env
│   ├── package.json
│   └── Dockerfile
├── frontend/
│   ├── src/
│   │   ├── api/
│   │   │   └── axios.js          # Axios instance with withCredentials: true
│   │   ├── context/
│   │   │   └── AuthContext.jsx   # Global auth state (user, loading, login, logout)
│   │   ├── components/
│   │   │   ├── PrivateRoute.jsx  # Redirect to /login if no session
│   │   │   ├── Navbar.jsx
│   │   │   ├── TaskCard.jsx
│   │   │   └── ProjectCard.jsx
│   │   ├── pages/
│   │   │   ├── Login.jsx
│   │   │   ├── Register.jsx
│   │   │   ├── Dashboard.jsx
```

```
│   │   │   ├── Projects.jsx
│   │   │   ├── ProjectDetail.jsx
│   │   │   └── Profile.jsx
│   │   └── main.jsx
│   ├── package.json
│   └── Dockerfile
└── docker-compose.yml
```

5. Database Schema

```
-- Users
CREATE TABLE users (
  id           SERIAL PRIMARY KEY,
  email        VARCHAR(255) UNIQUE NOT NULL,
  password_hash TEXT NOT NULL,
  display_name  VARCHAR(100) NOT NULL,
  role         VARCHAR(20) NOT NULL DEFAULT 'member', -- 'admin' | 'member'
  created_at    TIMESTAMPTZ DEFAULT NOW()
);

-- Projects
CREATE TABLE projects (
  id          SERIAL PRIMARY KEY,
  user_id     INTEGER NOT NULL REFERENCES users(id) ON DELETE CASCADE,
  name        VARCHAR(150) NOT NULL,
  description TEXT,
  color       CHAR(7) DEFAULT '#6366f1',             -- hex color
  status      VARCHAR(20) DEFAULT 'active',          -- 'active' | 'archived'
  created_at  TIMESTAMPTZ DEFAULT NOW()
);

-- Tasks
CREATE TABLE tasks (
  id          SERIAL PRIMARY KEY,
  project_id  INTEGER NOT NULL REFERENCES projects(id) ON DELETE CASCADE,
  title       VARCHAR(255) NOT NULL,
  description TEXT,
  priority    VARCHAR(10) DEFAULT 'medium',          -- 'low' | 'medium' | 'high'
  status      VARCHAR(20) DEFAULT 'todo',            -- 'todo' | 'in-progress' | 'don
  due_date    DATE,
  created_at  TIMESTAMPTZ DEFAULT NOW()
);

-- Activity Log
CREATE TABLE activity_log (
  id          SERIAL PRIMARY KEY,
  user_id     INTEGER NOT NULL REFERENCES users(id) ON DELETE CASCADE,
  action      VARCHAR(100) NOT NULL,                 -- e.g. 'completed task'
  entity_type VARCHAR(50),                           -- 'task' | 'project'
  entity_name VARCHAR(255),
  project_name VARCHAR(255),
  created_at  TIMESTAMPTZ DEFAULT NOW()
);
```

6. REST API Reference

Auth

Method	Endpoint	Auth required	Description
POST	`/auth/register`	No	Create new account
POST	`/auth/login`	No	Login, create session
POST	`/auth/logout`	Yes	Destroy session
GET	`/auth/me`	Yes	Get current user from session

Projects

Method	Endpoint	Auth required	Description
GET	`/projects`	Yes	List user's projects (Redis cached)
POST	`/projects`	Yes	Create project
PATCH	`/projects/:id`	Yes	Update project
DELETE	`/projects/:id`	Yes (owner or admin)	Delete project

Tasks

Method	Endpoint	Auth required	Description
GET	`/projects/:id/tasks`	Yes	List tasks in project
POST	`/projects/:id/tasks`	Yes	Create task
PATCH	`/projects/:id/tasks/:taskId`	Yes	Update task
DELETE	`/projects/:id/tasks/:taskId`	Yes	Delete task

Dashboard & Users

Method	Endpoint	Auth required	Description
GET	`/dashboard`	Yes	Summary stats + activity feed
GET	`/users`	Admin only	List all users
PATCH	`/users/:id/role`	Admin only	Change user role

7. Core Backend Code

`config/redis.js`

```
import { createClient } from 'redis';

const redisClient = createClient({ url: process.env.REDIS_URL });
redisClient.on('error', (err) => console.error('Redis error:', err));
await redisClient.connect();

export default redisClient;
```

`config/db.js`

```
import pg from 'pg';
const { Pool } = pg;

const pool = new Pool({ connectionString: process.env.DATABASE_URL });
export default pool;
```

`app.js`

```
import express from 'express';
import session from 'express-session';
import { RedisStore } from 'connect-redis';
import redisClient from './config/redis.js';
import cors from 'cors';
import helmet from 'helmet';

import authRoutes      from './routes/auth.js';
import projectRoutes   from './routes/projects.js';
import taskRoutes      from './routes/tasks.js';
import dashboardRoutes from './routes/dashboard.js';
import userRoutes      from './routes/users.js';

const app = express();

app.use(helmet());
app.use(express.json());
app.use(cors({
  origin: process.env.CLIENT_URL,
  credentials: true,
}));

app.use(session({
  store: new RedisStore({ client: redisClient }),
  secret: process.env.SESSION_SECRET,
  resave: false,
  saveUninitialized: false,
  cookie: {
    httpOnly: true,
```

```
    secure: process.env.NODE_ENV === 'production',
    sameSite: 'lax',
    maxAge: 1000 * 60 * 60 * 24, // 24h default
  },
}));

app.use('/auth',      authRoutes);
app.use('/projects',  projectRoutes);
app.use('/projects',  taskRoutes);
app.use('/dashboard', dashboardRoutes);
app.use('/users',     userRoutes);

app.listen(3000, () => console.log('Server running on port 3000'));
```

`middlewares/requireAuth.js`

```
export default function requireAuth(req, res, next) {
  if (!req.session?.userId)
    return res.status(401).json({ error: 'Not authenticated' });
  next();
}
```

`middlewares/requireRole.js`

```
export default function requireRole(role) {
  return (req, res, next) => {
    if (req.session?.role !== role)
      return res.status(403).json({ error: 'Forbidden' });
    next();
  };
}
```

`routes/auth.js`

```
import express from 'express';
import bcrypt from 'bcrypt';
import pool from '../config/db.js';
import requireAuth from '../middlewares/requireAuth.js';

const router = express.Router();

// Register
router.post('/register', async (req, res) => {
  const { email, password, display_name } = req.body;
  const hash = await bcrypt.hash(password, 12);
  const { rows } = await pool.query(
    'INSERT INTO users (email, password_hash, display_name) VALUES ($1,$2,$3) RETURN
    [email, hash, display_name]
  );
  res.status(201).json(rows[0]);
});
```

```
// Login
router.post('/login', async (req, res) => {
  const { email, password, rememberMe } = req.body;
  const { rows } = await pool.query('SELECT * FROM users WHERE email = $1', [email]);
  const user = rows[0];

  if (!user || !(await bcrypt.compare(password, user.password_hash)))
    return res.status(401).json({ error: 'Invalid credentials' });

  if (rememberMe) req.session.cookie.maxAge = 1000 * 60 * 60 * 24 * 7; // 7 days

  req.session.regenerate((err) => {
    if (err) return res.status(500).json({ error: 'Session error' });
    req.session.userId = user.id;
    req.session.role   = user.role;
    res.json({ id: user.id, email: user.email, display_name: user.display_name, ro
  });
});

// Logout
router.post('/logout', (req, res) => {
  req.session.destroy(() => {
    res.clearCookie('connect.sid');
    res.json({ message: 'Logged out' });
  });
});

// Get current user
router.get('/me', requireAuth, async (req, res) => {
  const { rows } = await pool.query(
    'SELECT id, email, display_name, role FROM users WHERE id = $1',
    [req.session.userId]
  );
  if (!rows[0]) return res.status(404).json({ error: 'User not found' });
  res.json(rows[0]);
});

export default router;
```

`routes/projects.js` (with Redis cache-aside)

```
import express from 'express';
import pool from '../config/db.js';
import redisClient from '../config/redis.js';
import requireAuth from '../middlewares/requireAuth.js';

const router = express.Router();

router.get('/', requireAuth, async (req, res) => {
  const cacheKey = `user:${req.session.userId}:projects`;
  const cached = await redisClient.get(cacheKey);
  if (cached) return res.json(JSON.parse(cached));

  const { rows } = await pool.query(
```

```
    'SELECT * FROM projects WHERE user_id = $1 ORDER BY created_at DESC',
    [req.session.userId]
  );
  await redisClient.setEx(cacheKey, 300, JSON.stringify(rows)); // cache 5 min
  res.json(rows);
});

router.post('/', requireAuth, async (req, res) => {
  const { name, description, color } = req.body;
  const { rows } = await pool.query(
    'INSERT INTO projects (user_id, name, description, color) VALUES ($1,$2,$3,$4) I
    [req.session.userId, name, description, color]
  );
  await redisClient.del(`user:${req.session.userId}:projects`); // invalidate cache
  res.status(201).json(rows[0]);
});

router.patch('/:id', requireAuth, async (req, res) => {
  const { name, description, color, status } = req.body;
  const { rows } = await pool.query(
    'UPDATE projects SET name=$1, description=$2, color=$3, status=$4 WHERE id=$5 AN
    [name, description, color, status, req.params.id, req.session.userId]
  );
  if (!rows[0]) return res.status(404).json({ error: 'Project not found' });
  await redisClient.del(`user:${req.session.userId}:projects`);
  res.json(rows[0]);
});

router.delete('/:id', requireAuth, async (req, res) => {
  await pool.query(
    'DELETE FROM projects WHERE id=$1 AND user_id=$2',
    [req.params.id, req.session.userId]
  );
  await redisClient.del(`user:${req.session.userId}:projects`);
  res.status(204).send();
});

export default router;
```

routes/dashboard.js

```
router.get('/', requireAuth, async (req, res) => {
  const userId = req.session.userId;
  const cacheKey = `user:${userId}:dashboard`;
  const cached = await redisClient.get(cacheKey);
  if (cached) return res.json(JSON.parse(cached));

  const [projects, dueToday, completedWeek, overdue, activity] = await Promise.all(
    pool.query("SELECT COUNT(*) FROM projects WHERE user_id=$1 AND status='active'",
    pool.query("SELECT COUNT(*) FROM tasks t JOIN projects p ON t.project_id=p.id WH
    pool.query("SELECT COUNT(*) FROM tasks t JOIN projects p ON t.project_id=p.id WH
    pool.query("SELECT COUNT(*) FROM tasks t JOIN projects p ON t.project_id=p.id WH
    pool.query('SELECT * FROM activity_log WHERE user_id=$1 ORDER BY created_at DESC
  ]);
```

```
  const data = {
    activeProjects:   parseInt(projects.rows[0].count),
    tasksDueToday:    parseInt(dueToday.rows[0].count),
    completedThisWeek: parseInt(completedWeek.rows[0].count),
    overdueTasks:     parseInt(overdue.rows[0].count),
    recentActivity:   activity.rows,
  };

  await redisClient.setEx(cacheKey, 60, JSON.stringify(data)); // cache 60s
  res.json(data);
});
```

8. Frontend Core Code

`api/axios.js`

```
import axios from 'axios';

const api = axios.create({
  baseURL: import.meta.env.VITE_API_URL || 'http://localhost:3000',
  withCredentials: true, // send session cookie on every request
});

export default api;
```

`context/AuthContext.jsx`

```
import { createContext, useContext, useState, useEffect } from 'react';
import api from '../api/axios';

const AuthContext = createContext(null);

export function AuthProvider({ children }) {
  const [user, setUser]       = useState(null);
  const [loading, setLoading] = useState(true);

  useEffect(() => {
    api.get('/auth/me')
      .then(res => setUser(res.data))
      .catch(() => setUser(null))
      .finally(() => setLoading(false));
  }, []);

  const login = async (email, password, rememberMe) => {
    const res = await api.post('/auth/login', { email, password, rememberMe });
    setUser(res.data);
  };

  const logout = async () => {
    await api.post('/auth/logout');
    setUser(null);
```

```
  };

  return (
    <AuthContext.Provider value={{ user, loading, login, logout }}>
      {children}
    </AuthContext.Provider>
  );
}

export const useAuth = () => useContext(AuthContext);
```

components/PrivateRoute.jsx

```
import { Navigate } from 'react-router-dom';
import { useAuth } from '../context/AuthContext';

export default function PrivateRoute({ children }) {
  const { user, loading } = useAuth();
  if (loading) return <div>Loading...</div>;
  return user ? children : <Navigate to="/login" replace />;
}
```

9. Docker Compose

```
services:
  backend:
    build: ./backend
    ports:
      - "3000:3000"
    env_file: ./backend/.env
    depends_on:
      - postgres
      - redis

  frontend:
    build: ./frontend
    ports:
      - "5173:5173"
    environment:
      - VITE_API_URL=http://localhost:3000

  postgres:
    image: postgres:16-alpine
    environment:
      POSTGRES_DB: devboard
      POSTGRES_USER: devboard_user
      POSTGRES_PASSWORD: devboard_pass
    volumes:
      - pgdata:/var/lib/postgresql/data
      - ./backend/src/db/schema.sql:/docker-entrypoint-initdb.d/schema.sql
    ports:
      - "5432:5432"
```

```
  redis:
    image: redis:7-alpine
    ports:
      - "6379:6379"

volumes:
  pgdata:
```

10. Environment Variables

`backend/.env`

```
DATABASE_URL=postgresql://devboard_user:devboard_pass@postgres:5432/devboard
REDIS_URL=redis://redis:6379
SESSION_SECRET=change_this_to_a_long_random_string_minimum_32_chars
CLIENT_URL=http://localhost:5173
NODE_ENV=development
PORT=3000
```

11. Auth Flow Diagram

```
React                Express              Redis              PostgreSQL
  |                     |                    |                    |
  |-- POST /auth/login -->|                    |                    |
  |                     |-- SELECT user ---------------------------------->|
  |                     |<------------------------------ user row -------|
  |                     |-- bcrypt.compare()  |                    |
  |                     |-- session.regenerate()                   |
  |                     |-- SET session:{sid} --> userId, role     |
  |<-- Set-Cookie: sid --|                    |                    |
  |                     |                    |                    |
  |-- GET /projects ----->|                    |                    |
  |   (cookie auto-sent) |-- GET session:{sid} ->|                 |
  |                     |<--- userId, role ----|                   |
  |                     |-- GET user:X:projects -->|               |
  |                     |<--- cache HIT ----------|              |
  |<-- 200 projects data--|                    |                    |
  |                     |                    |                    |
  |-- POST /auth/logout ->|                    |                    |
  |                     |-- DEL session:{sid} -->|                 |
  |<-- 200 / clear cookie|                    |                    |
```

12. Redis Usage Summary

Key pattern	What is stored	TTL	Invalidated by
`session:{sid}`	userId, role	24h / 7d	Logout
`user:{id}:projects`	Project list array	5 min	Any project mutation
`user:{id}:dashboard`	Dashboard stats object	60 sec	TTL expiry

13. Resume Description

> **DevBoard** — Full-Stack Project Management App | [GitHub Link] | [Live Demo Link]
> Built with Node.js/Express, PostgreSQL, Redis, and React. Features session-based authentication with Redis-backed session store, role-based access control (admin/member), cache-aside pattern for project data with manual invalidation, and RESTful API design. Containerized with Docker Compose for one-command local setup.

14. Recommended Build Order

1. Docker Compose + PostgreSQL schema + Redis setup
2. Express app.js + session config
3. Auth routes (register, login, logout, me)
4. Projects routes (with Redis caching)
5. Tasks routes (CRUD)
6. Dashboard route (aggregation queries)
7. React: AuthContext + PrivateRoute + Login page
8. React: Projects page + Project detail page
9. React: Dashboard page + Profile page
10. Polish: error handling, loading states, empty states
11. README with architecture diagram, setup instructions, live demo link